T07 CLAUDE.md Best Practices

Technical Intermediate

Writing effective instructions that Claude reads every session

What to include

An effective CLAUDE.md contains three essential blocks: the tech stack, build/test commands, and critical conventions. Anything Claude can infer from the code itself (folder structure, detected framework, package manager) does not need to be documented.

```
## Stack
Next.js 14, TypeScript, PostgreSQL/Prisma, pnpm

## Commands
- `pnpm dev`  — start the server
- `pnpm test`  — run tests
- `pnpm build` — production build

## Critical conventions
- Always pnpm, never npm or yarn
- Functional components only
- kebab-case files (my-component.tsx)
```

What NOT to include

Avoid	Reason
Long narrative documentation	Dilutes critical rules
Historical context (“In 2023 we decided…”)	Does not help Claude act
References to entire files	Use `@path/file` instead
Rules Claude already follows	Unnecessary noise, token cost

Recommended size

Keep instructions concise and review their effect on real tasks. About 200 lines is a practical starting point, not a demonstrated boundary for model consistency.

Practical rule: if Claude makes the same mistake twice, add the rule. Do not document everything preemptively.

Hierarchical structure

```
~/.claude/CLAUDE.md          (global, all projects)
/project/CLAUDE.md           (project, committed)
/project/CLAUDE.local.md     (personal, add to .gitignore)
```

In a monorepo, Claude loads the root CLAUDE.md then merges the CLAUDE.md from the active subfolder. Child rules complement parent rules without overriding them.

Explicit imports with @

```
# In CLAUDE.md
@docs/conventions/coding-standards.md
@docs/conventions/architecture.md
```

Imports in CLAUDE.md load with that instruction file. Unscoped rules load generally; rules with `paths` frontmatter load when matching files are relevant. Use a skill when the complete workflow should load only on invocation.

Recommended modular architecture

```
.claude/
├── CLAUDE.md          (index, < 100 lines)
└── rules/
    ├── testing.md     (coverage thresholds)
    ├── security.md    (security invariants)
    └── api.md         (REST conventions)
```

CLAUDE.md stays readable at a glance. Detailed rules live in `.claude/rules/`; scope file-specific instructions with `paths`.